

作业指导手册

MobileNetV2 + Faster R-CNN 火焰 / 烟雾目标检测项目

从“完全看不懂”到完成训练、评估、云端实验与 GitHub 交付

本手册适合谁 零基础或刚开始接触目标检测的学习者。重点不是背代码，而是知道每一步为什么做、出错时先查哪里。

项目主线

数据 → 环境 → 模型 → 训练 → mAP → 曲线 → 云服务器 → GitHub / PPT

阅读说明：先抓主线，再处理细节

这不是一份要求你背诵所有 API 的教材，而是一份面向初学者的项目作业手册。你可以把它当作以后做类似项目时的检查清单：先确认任务属于分类还是检测，再确认数据格式、环境、模型输入输出，最后才开始长时间训练。

最重要的学习标准 能讲清数据如何进入模型、模型如何输出框和类别、mAP 如何得到、结果文件在哪里；不要求你独立默写 Faster R-CNN 或手推所有矩阵。

本项目最后完成了什么

项目项	本次实际方案
任务	火焰 fire 与烟雾 smoke 的目标检测
骨干网络	MobileNetV2，负责提取图像特征
检测框架	Faster R-CNN，负责候选区域、分类与边界框回归
数据标签	YOLO 格式：class x_center y_center width height
训练	PyTorch + Torchvision + CUDA，100 epochs，batch size 8，AdamW，学习率 1e-4
评估	自定义 IoU / AP / mAP 评估脚本
交付	训练权重、日志、评估 JSON、曲线图、GitHub 仓库、PPT

一页流程图

建议把这条链路写在笔记第一页



第一章 先把任务理解对

1.1 MobileNetV2 不是完整目标检测器

MobileNetV2 原本更常被当作分类网络或骨干网络。它擅长从图片中提取特征，但单独使用时通常只输出类别概率，并不会给出目标框。

名称	它负责什么	本项目中的位置
MobileNetV2	提取边缘、纹理、形状等特征	backbone
Faster R-CNN	根据特征提出候选区域，并预测类别与边界框	检测框架
mAP	评价预测框与真实框的整体检测效果	最终指标

不要混淆 “MobileNetV2 + Faster R-CNN”才是本项目的完整检测模型；“MobileNetV2 单独分类”不能自然产生目标检测 mAP。

1.2 三个问题必须先回答

- 输入是什么：整张图片，还是裁剪后的目标？
- 输出是什么：一个类别，还是多个“类别 + 边界框”？
- 指标是什么：accuracy，还是 IoU / AP / mAP？

本项目的答案是：整张图片输入；输出多个火焰/烟雾框及类别；使用 mAP 评价。

第二章 数据集检查：先确认再训练

2.1 目录结构

最小可用的目标检测数据结构

```

data/
├── train/
│   ├── images/
│   └── labels/
├── val/
│   ├── images/
│   └── labels/
└── test/
    ├── images/
    └── labels/
    
```

目录	作用	初学者要点
train	训练模型参数	样本最多，训练时反复使用
val	训练过程中验证和选模型	本项目每个 epoch 都用它评估
test	最终独立测试或展示	云端重新训练时不一定必须上传，但完整交付建议保留

2.2 YOLO 标签格式

每行一个目标；坐标和宽高都归一化到 0~1

class_id	x_center	y_center	width	height
0	0.50	0.40	0.20	0.30

字段	含义	本项目
class_id	类别编号	0 = fire, 1 = smoke
x_center / y_center	框中心相对图片左上角的位置	归一化坐标
width / height	框宽高占图片宽高的比例	归一化尺寸

代码读取标签后，需要把 YOLO 的中心点格式转换成 Faster R-CNN 需要的 [xmin, ymin, xmax, ymax] 像素坐标。类别编号还要整体加 1，因为 Faster R-CNN 通常把 0 留给 background。

2.3 数据集必查清单

- 类别映射是否明确：0 是 fire，1 是 smoke；不要凭文件名猜。
- 图片和标签是否同名：A.jpg 应对应 A.txt。
- 标签是否有五列；是否存在 NaN、负数、超过 1 的坐标。
- 是否有大量无目标图片；空标注不是一定错误，但模型代码必须能处理。
- 是否有零宽或零高框；YOLO 工具可能较宽松，Torchvision 检测模型通常会直接报错。

这次实际踩过的坑 数据中存在背景图和异常框。空框必须整理成形状为 [0, 4] 的张量；非法框要在 Dataset 中过滤，而不是等模型训练时报错。

第三章 环境搭建：先让 GPU 能被 PyTorch 看到

3.1 本地环境的正确检查顺序

1. 创建并激活 conda 环境，例如 fire_detection。
2. 安装与显卡兼容的 PyTorch / Torchvision。
3. 进入 Python 解释器后，再运行 Python 代码。
4. 确认 torch.cuda.is_available() 为 True，并查看显卡名称。

命令行与 Python 代码要分清

```
conda activate fire_detection
python

import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
print(torch.version.cuda)
```

看到什么	说明	该输入什么
(fire_detection) C:\...>	在终端 / CMD	pip、conda、python train.py 等命令
>>>	在 Python 解释器	import torch、print(...) 等 Python 代码

这次实际踩过的坑 把 import torch、print(...) 直接输入 Windows 命令行会出现“import 不是命令”；把 pip install 输入 Python 解释器会出现 SyntaxError。

3.2 CUDA 版本不要被显示值绕晕

nvidia-smi 显示的 CUDA Version 往往表示驱动支持的最高 CUDA 版本，不一定等于当前 PyTorch 使用的 CUDA 运行时。真正要检查的是 torch.cuda.is_available()、torch.version.cuda 和显卡名称是否正常。

3.3 云服务器配置

本项目后期使用云端 RTX 5090 重新跑 100 epochs，batch size 调整为 8。云端训练的核心不是“显存越大越好”，而是算力、数据读取和 batch size 的综合吞吐。

云服务器最小启动流程

```
nvidia-smi
cd /root/mobilenetv2_project
python train.py
```

上传到云端训练至少需要：代码文件、train 数据、val 数据。test 数据用于最终测试或展示，可按任务需要决定是否上传。重新训练时不必上传旧的 .pth。

第四章 模型：MobileNetV2 + Faster R-CNN

4.1 模型分工

```

图片
↓
MobileNetV2.features: 提取特征
↓
Faster R-CNN
├─ RPN: 提出候选区域
├─ RoI: 对候选区域进一步处理
├─ 分类: fire / smoke / background
├─ 回归: 修正边界框
↓
多个预测框 + 类别 + 置信度

```

项目代码里使用 num_classes=3，不是 2：0 留给背景，1 对应 fire，2 对应 smoke。这个编号转换是检测模型最容易出错的地方之一。

4.2 为什么这套模型训练慢

MobileNetV2 本身是轻量 backbone，但 Faster R-CNN 是两阶段检测器：先生成候选区域，再分类和回归。它通常比 YOLO 这类一阶段检测器慢很多。实际项目中要把“backbone 轻量”和“整体检测器运行快”区分开。

不要把一次跑通的 YOLO 结果当成本项目结果 前期 YOLOv8n 只用于验证数据、CUDA 和训练流程；正式交付结果来自 MobileNetV2 backbone + Faster R-CNN。

第五章 训练：先跑小验证，再跑长实验

5.1 推荐训练顺序

- 5. 先用很少 epochs 或少量数据验证：路径、Dataset、模型、loss、GPU 是否正常。
- 6. 确认第一个 batch 能完成 forward 和 backward。
- 7. 确认第一个 epoch 能结束，验证集能跑完，日志能写入。
- 8. 最后再按项目要求跑 100 epochs。

5.2 本次正式实验参数

参数	值	说明
epochs	100	完整训练轮数
batch size	8	云端 RTX 5090 版本
optimizer	AdamW	优化器
learning rate	1e-4	固定学习率
num_workers	0	优先保证 Windows / 兼容性；Linux 可再测试提高

5.3 日志要记录什么

- 每个 epoch 的 train_loss。
- 每个 epoch 的 learning_rate。
- 每个 epoch 的验证 mAP、precision、recall。
- 权重文件保存路径，以及最后一次独立评估结果。

重要的真实性提醒 这次最终 training_log 中 val_loss、cls_loss、box_loss 有部分被填为 0，并不代表真实损失为 0。报告中不要把这些 0 画成真实验证曲线；真实可靠的核心指标是 train_loss、独立评估 mAP 和按类别 AP。

第六章 常见报错与排查表

错误现象	常见原因	正确处理
Dataset images not found	data.yaml 或相对路径指错；代码目录和数据目录层级不同	先画目录树，再用绝对路径或 Path(__file__) 计算 DATA_ROOT；不要凭感觉改目录

错误现象	常见原因	正确处理
import 不是命令	把 Python 代码输入了 CMD	先输入 python，出现 >>> 后再运行 Python 代码
SyntaxError: pip install	把终端命令输入了 Python 解释器	退出 >>>，回到 conda 终端再执行 pip
Windows multiprocessing spawn	DataLoader workers 多进程与脚本入口冲突	加 if __name__ == '__main__'; 初学者先设 num_workers=0
boxes shape [0]	空标注被转成一维空 tensor	使用 torch.zeros((0, 4)), labels 使用 torch.zeros((0,), dtype=torch.int64)
positive height and width	某个框 x1=x2 或 y1=y2	在 Dataset 中过滤 x2<=x1 或 y2<=y1 的非法框
Anchors should be Tuple[Tuple[int]]	Faster R-CNN backbone 特征图与 anchor 配置不匹配	明确单特征图配置、AnchorGenerator 和 MultiScaleRoIAlign 的 featmap_names
evaluate_model import / keyword error	train.py 与 evaluate.py 的函数名或参数接口不一致	先统一函数签名和返回字典，再开始长时间训练
KeyError: val_loss / mAP50	训练脚本期待的字段名与评估脚本实际返回字段不同	统一字段命名；不要一边写 mAP50，一边返回 mAP@0.5
GitHub push connection reset	Git 代理配置不对，或直连 GitHub 不稳定	检查 git proxy；使用当前实际代理端口；先 git ls-remote 测试
GitHub 拒绝 .pth	模型权重超过普通 GitHub 文件限制	用 .gitignore 忽略 *.pth；单独网盘/云盘交付权重

第七章 云服务器操作清单

7.1 上传前

- 确认本地代码已经在小规模实验中跑通。
- 确认云端目录层级，例如 /root/data/train、/root/data/val、/root/mobilenetv2_project。
- 代码中使用 DATA_ROOT = PROJECT_DIR.parent / 'data' 时，PROJECT_DIR 应是 /root/mobilenetv2_project。
- 不要把旧的 YOLO runs、旧结果和不需要的 .pth 混入云端训练目录。

7.2 启动前

一组快速健康检查

```
nvidia-smi
python -c "import torch; print(torch.cuda.is_available());
print(torch.cuda.get_device_name(0)); print(torch.version.cuda)"
cd /root/mobilenetv2_project
python train.py
```


7.3 长时间训练

- 建议使用 screen 或 tmux，避免网页终端断开导致训练停止。
- 训练中定期查看 GPU 利用率、显存、epoch 和日志文件。
- 每个 epoch 保存权重或日志，避免长任务中断后全部丢失。
- 不要在训练已经正常运行时随意改 batch、学习率或代码。

7.4 训练完成后

生成曲线和最终独立评估结果

```
python plot.py
python evaluate.py --device cuda --batch-size 8 --data-root /root/data
ls -lh
```

第八章 评估指标：知道数字在说什么

指标	通俗含义	本次最终结果
mAP@0.5	IoU 至少达到 0.5 时，各类别 AP 的平均值	0.698624
mAP@0.5:0.95	IoU 从 0.5 到 0.95 多个阈值综合平均，更严格	0.362444
Fire AP@0.5	火焰类别的 AP	0.781061
Smoke AP@0.5	烟雾类别的 AP	0.616186
推理速度	单张图平均推理耗时，受硬件、batch、实现影响	约 5.80 ms/image

烟雾 AP 低于火焰 AP 是合理现象：烟雾边界模糊、透明、形态变化大，定位比火焰更难。不要只看一个总 mAP，也要看按类别 AP。

最终值以谁为准 以最后一次独立运行 evaluate.py 生成的 evaluation_results.json 为准；训练过程中某个 epoch 打印的 mAP 只是当时的中间值，不能替代最终评估。

第九章 曲线与结果文件

9.1 结果文件各自是什么

文件	用途	是否适合直接展示
mobilenetv2_fasterrcnn.pth	模型参数权重	不能双击打开；用于加

文件	用途	是否适合直接展示
		载模型
training_log.json	每轮训练记录	可用于分析和画图
evaluation_results.json	最终验证集指标	适合报告引用
training_curve.png	训练过程可视化	适合 PPT
train.py / evaluate.py / dataset.py / model.py / plot.py	复现实验的代码	适合 GitHub

9.2 曲线中的常见误区

- 固定学习率时，learning rate 曲线是一条水平线，不是错误。
- 如果 val_loss、cls_loss、box_loss 没有真实统计，不要为了“像示例图”而填 0 或伪造曲线。
- 曲线应该标明数据来源、epoch 数、指标定义和是否为训练过程或独立评估。
- 最终 mAP 图和训练过程 mAP 图不要混为一谈。

第十章 GitHub 整理与上传

10.1 推荐仓库结构

```
fire-smoke-detection-mobilenetv2-fasterrcnn/
├── README.md
├── requirements.txt
├── .gitignore
├── src/
│   ├── train.py
│   ├── evaluate.py
│   ├── dataset.py
│   ├── model.py
│   └── plot.py
├── results/
│   ├── training_curve.png
│   ├── training_log.json
│   └── evaluation_results.json
└── weights/
    └── README.md
```

10.2 不建议放进公开仓库

- 完整 train / val 图片数据集：体积大，且可能涉及数据集许可问题。
- 模型权重 .pth：通常超过 GitHub 普通文件限制。
- __pycache__、*.pyc、runs、临时目录。

- 包含个人路径、密码、代理地址或服务器密钥的文件。

10.3 Git 命令模板

第一次上传前，先用 `git status` 确认没有 `.pth` 和数据集

```
cd C:\Users\yangd\Desktop\fire-smoke-detection-mobilenetv2-fasterRCNN
git init
git add .
git status
git config --global user.name "你的名字"
git config --global user.email "你的GitHub 邮箱"
git commit -m "Initial commit: fire smoke detection"
git branch -M main
git remote add origin https://github.com/用户名/仓库名.git
git push -u origin main
```

这次实际踩过的坑 Git 的 LF/CRLF warning 通常不是错误；commit 前需要配置 `user.name` 和 `user.email`；Git 使用的代理端口必须和本机代理真实端口一致。

给项目经理分享网页浏览链接时，通常发不带 `.git` 的仓库地址；带 `.git` 的地址主要用于 `git clone`。

第十一章 PPT 与项目交付

11.1 PPT 推荐结构

9. 项目背景与目标
10. 数据集与类别
11. 整体技术路线
12. MobileNetV2 + Faster R-CNN 模型结构
13. 训练环境与超参数
14. 训练曲线
15. 最终 mAP 与按类别 AP
16. 检测效果或可视化结果
17. 问题与解决过程
18. 总结与后续优化

11.2 给项目经理的最小交付包

材料	建议是否提供	用途
GitHub 仓库链接	是	查看代码、README 和实验结果
PPT	是	汇报项目背景、方案和结果
训练权重 .pth	按要求	复现推理或后续部署；可用网盘单独传
training_log.json / evaluation_results.json	是	核对训练过程和最终指标
training_curve.png	是	展示训练过程
完整数据集	按要求	通常不随代码仓库上传

11.3 交付说明模板

项目已完成，代码及实验结果已整理到 GitHub。

模型：MobileNetV2 backbone + Faster R-CNN

任务：fire / smoke 目标检测

训练：100 epochs, batch size 8, AdamW, learning rate 1e-4

最终验证集 mAP@0.5: 0.698624

最终验证集 mAP@0.5:0.95: 0.362444

代码仓库：

<https://github.com/用户名/仓库名>

第十二章 初学者最终检查清单

A. 开始训练前

- ☐ 我确认这是目标检测，不是单纯分类。
- ☐ 我知道 0=fire、1=smoke。
- ☐ train / val / test 的目录和 images / labels 都存在。
- ☐ 随机检查过图片和同名标签。
- ☐ 我能用一句话解释 YOLO 标签五列。
- ☐ torch.cuda.is_available() 为 True。
- ☐ train.py、evaluate.py 的函数名、参数名和返回字段已经统一。
- ☐ 先用小规模运行验证过第一个 batch / 第一个 epoch。

B. 训练过程中

- ☐ GPU 利用率和显存占用正常。
- ☐ loss 能正常计算，不出现 NaN。
- ☐ epoch 能顺利结束，验证集能跑完。
- ☐ training_log.json 会逐轮写入。
- ☐ 不在长任务中途随意改代码或参数。

C. 训练结束后

- ☐ 权重 .pth 已保存。
- ☐ 独立运行 evaluate.py，得到 evaluation_results.json。
- ☐ 以最终 JSON 为准记录 mAP 和按类别 AP。
- ☐ 运行 plot.py，检查曲线是否有真实数据来源。
- ☐ 更新 PPT 和 README，删除旧的 10 epoch 结果。
- ☐ GitHub 中没有数据集、密码、代理配置和大模型权重。

第十三章 这次项目的复盘结论

这次项目真正练到的，不只是“把模型跑起来”，而是一条完整的 AI 项目链路：需求理解、数据核对、环境配置、模型组合、训练调试、指标评估、可视化、云端算力、工程整理和对外交付。

你在过程中遇到的很多问题并不低级：路径层级、类别编号、空标注、非法框、Windows 多进程、脚本接口、评估字段、代理和大文件，都是实际项目中常见的工程问题。真正重要的是以后形成固定习惯：每次出错，先看完整报错的第一处关键原因，再检查路径、数据形状、函数签名和运行环境，不要一上来重写所有代码。

最后记住这句话 先跑通最小闭环，再做长训练；先保证指标真实，再追求图表漂亮；先统一代码接口，再让服务器跑一整晚。

附录：常用命令速查

```
# conda / GPU
conda activate fire_detection
nvidia-smi

# 运行训练 / 评估 / 绘图
python train.py
python evaluate.py --device cuda --batch-size 8 --data-root /root/data
python plot.py

# GitHub
git status
git add .
git commit -m "update experiment results"
git push

# 检查当前 Git 代理
git config --global --get http.proxy
git config --global --get https.proxy
```

建议把本手册和项目 README 放在同一个项目资料目录里。以后做新的图像分类、目标检测或分割项目，可以复制这份清单，只替换任务、数据集、模型和指标。